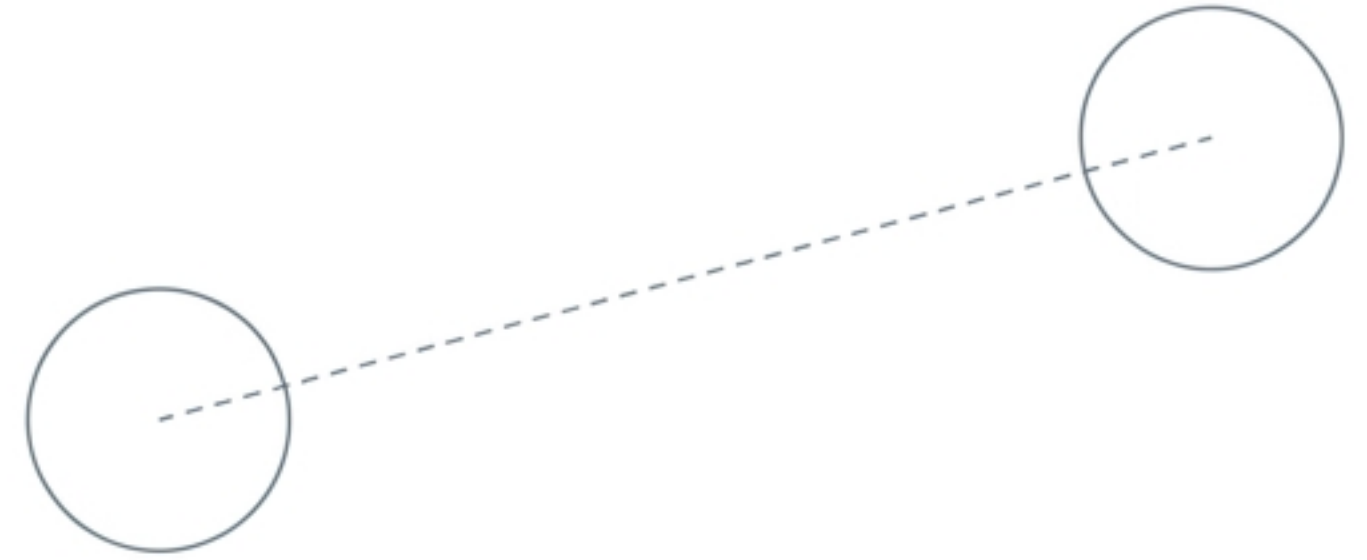


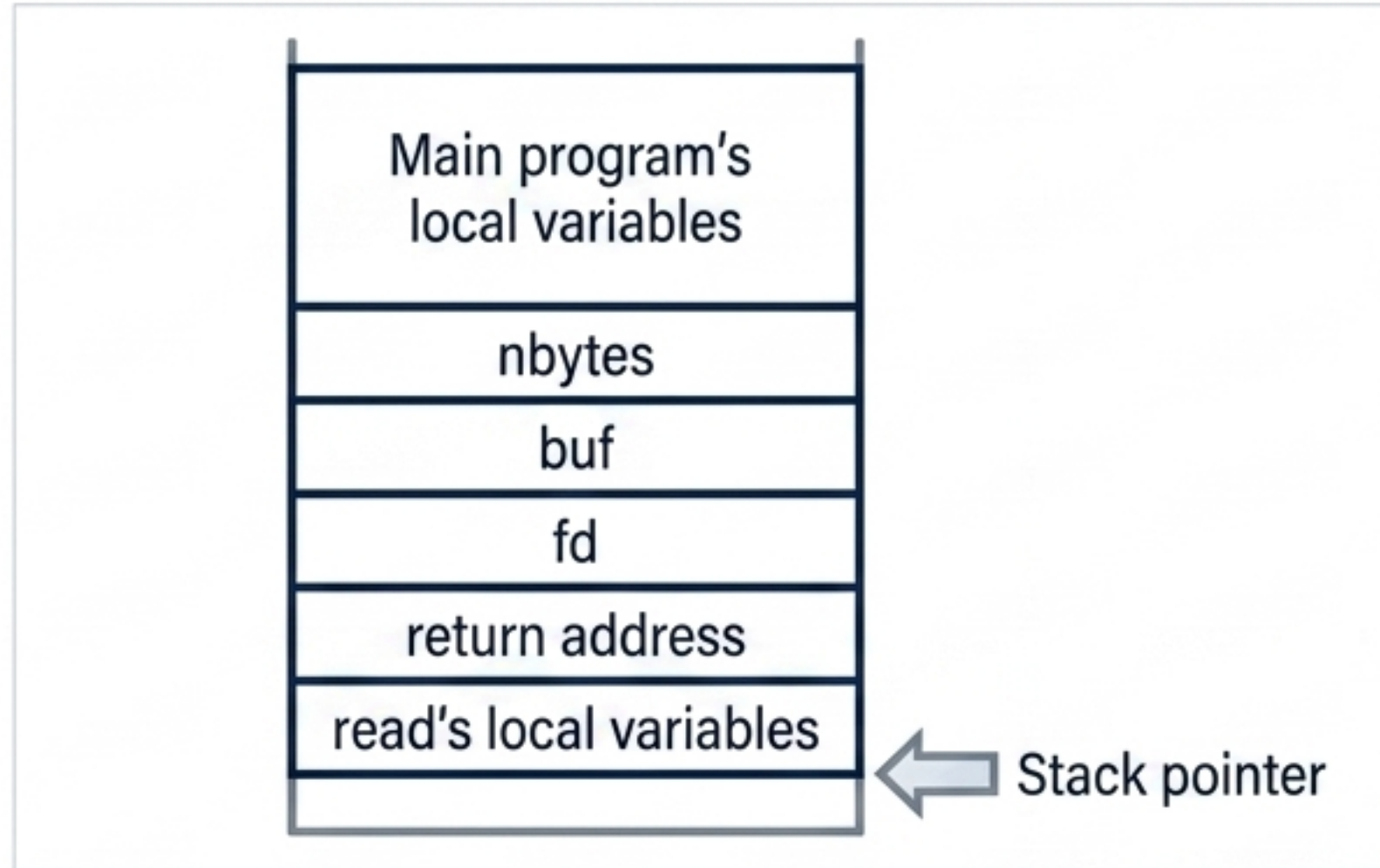


The Illusion of Proximity

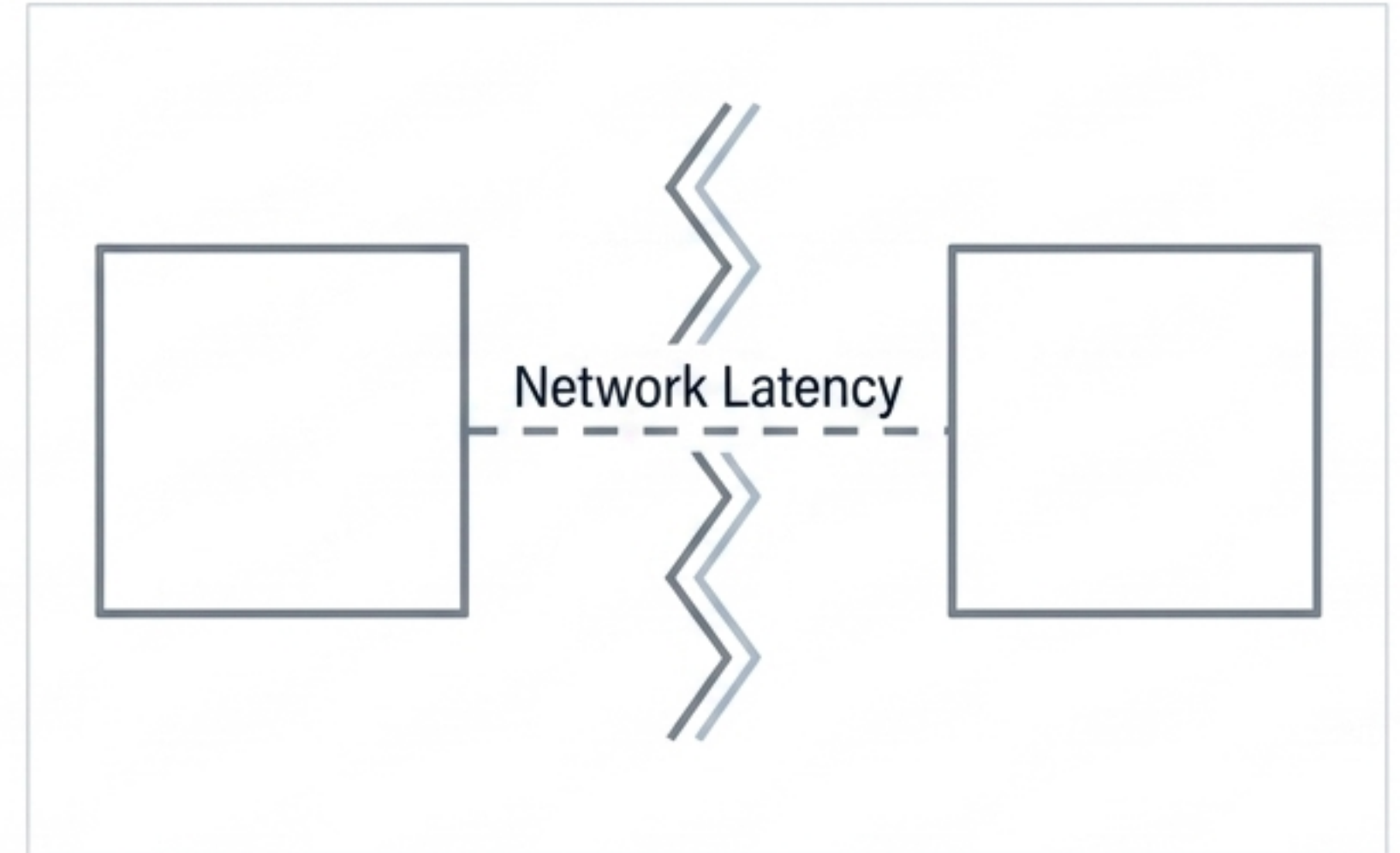
Mastering RPC and RMI in Distributed Systems

The Distributed Chasm

Local procedure calls rely on a shared memory stack. Distributed systems do not.



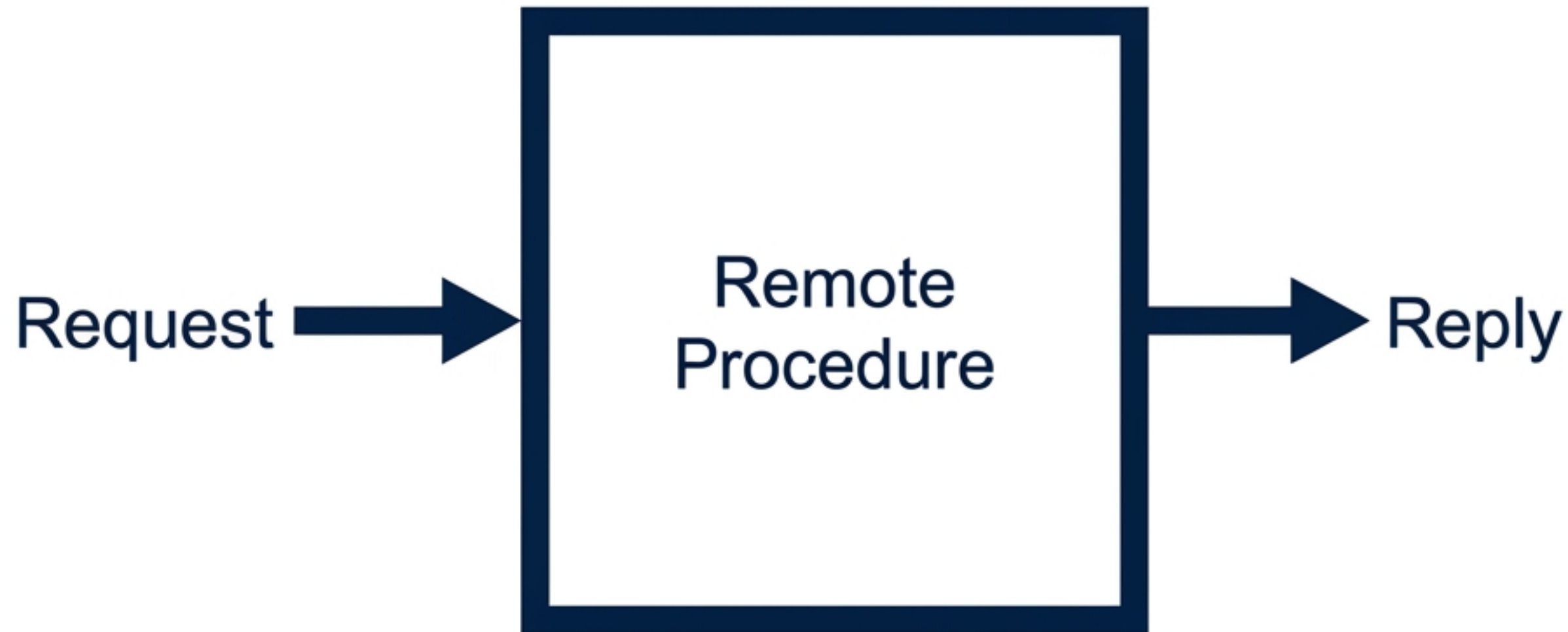
Local Execution: Instant, reliable, shared state.



Remote Execution: Network latency, isolated memory, representation mismatches.

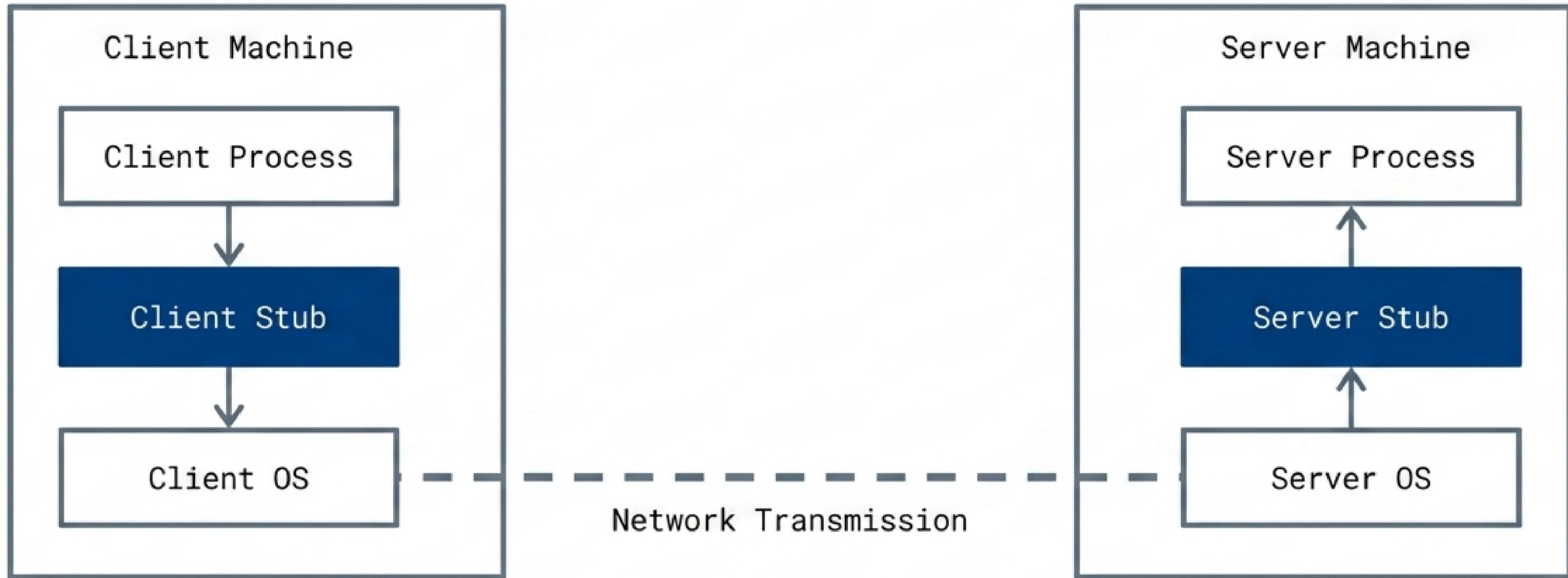
The Goal: Bridge the physical network gap seamlessly.

The RPC Philosophy



- Leverages the familiar procedure-call model application developers already know.
- Treats remote execution as an isolated black box.
- Core Promise: Communication between caller and callee is entirely hidden by the procedure-call mechanism.

Anatomy of an RPC

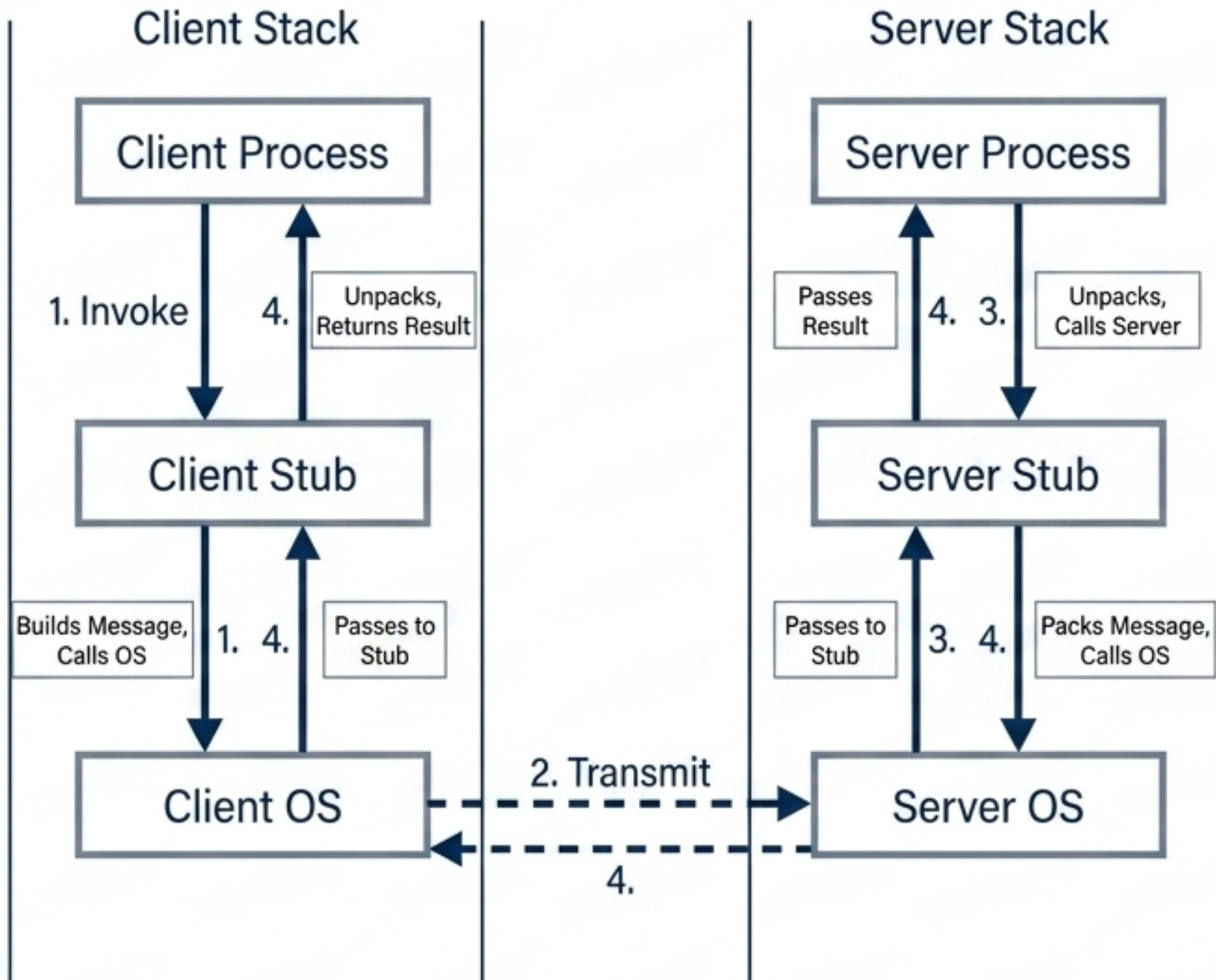


Client Stub: Acts as the local proxy, packs the request, and calls the local OS.

Server Stub: Unpacks the request and invokes the actual server implementation.

The OS Layer: Handles the raw network transmission between machines.

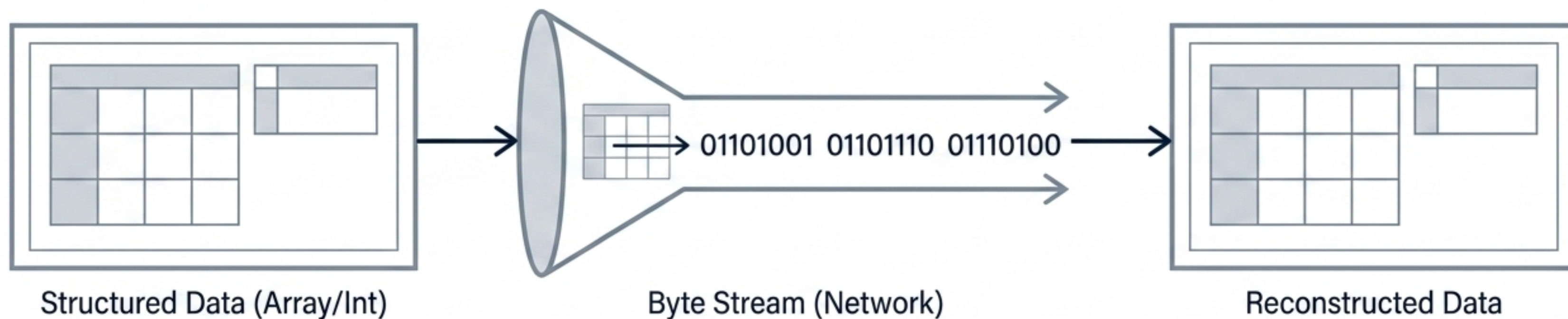
The Execution Flow



- 1. Invoke:** Client calls stub. Stub builds message and calls local OS.
- 2. Transmit:** Client OS sends message across network to Remote OS.
- 3. Unpack & Execute:** Remote OS passes message to Server Stub. Stub unpacks parameters and calls the server to do work.
- 4. Return:** Server passes result to Stub. Stub packs message, calls OS, and transmits back to Client Stub to unpack.

The Marshaling Challenge

Wrapping a parameter means transforming values into a flat sequence of bytes.



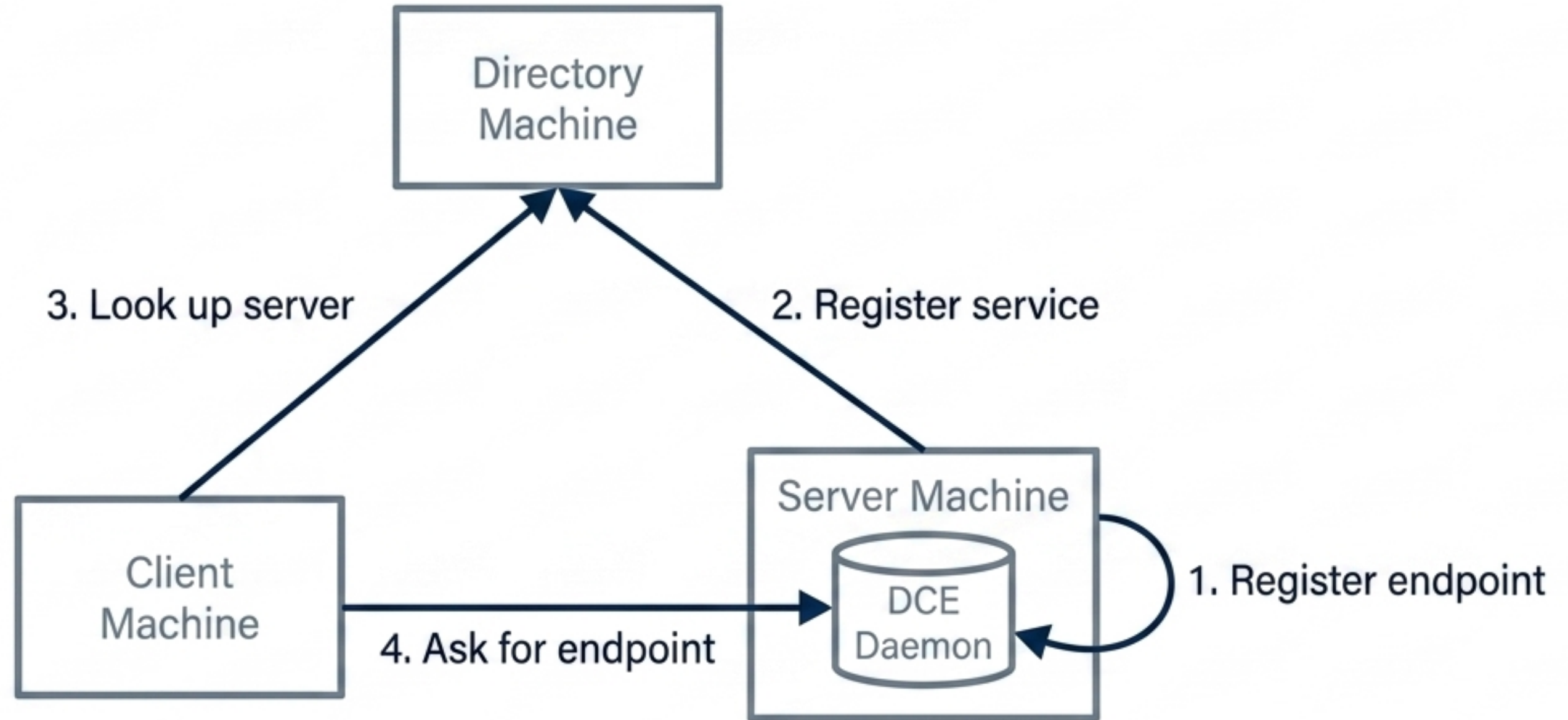
- Relies strictly on copy-in/copy-out semantics.
- Global references cannot be passed—full access transparency is impossible without remote reference mechanisms.



Data Representation Mismatch: Must resolve different machine data encoding rules, such as byte ordering differences between architectures.

Client-to-Server Binding

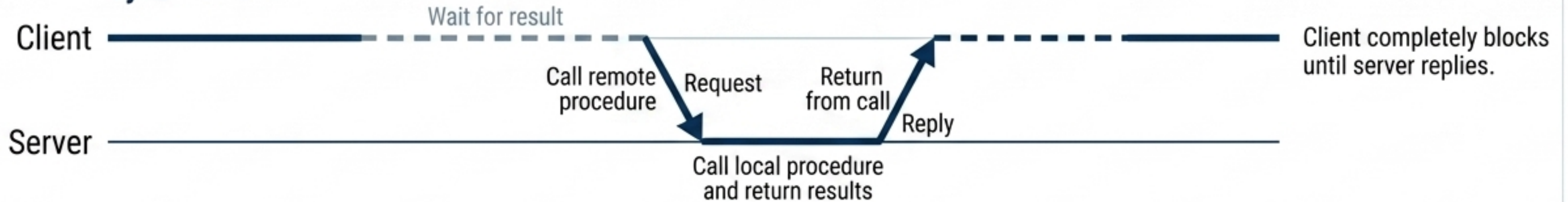
Before invoking, the client must locate the server and its specific endpoint.



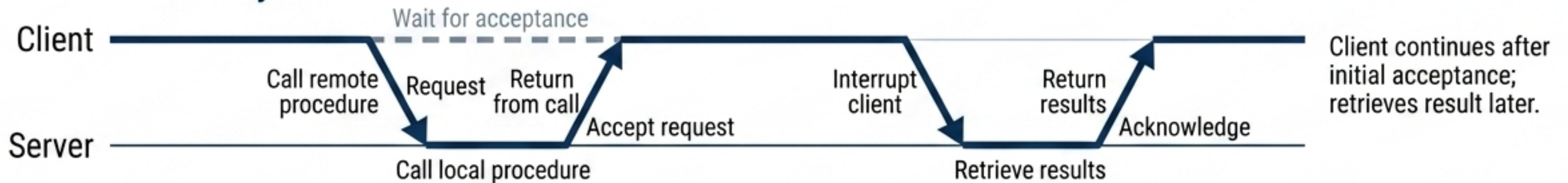
- Step 1: Server registers its endpoint with a local DCE daemon.
- Step 2: Server registers its service with a Directory Machine.
- Step 3: Client asks Directory for the server, then queries the daemon for the exact endpoint.

Breaking the Wait

Synchronous



Deferred Synchronous

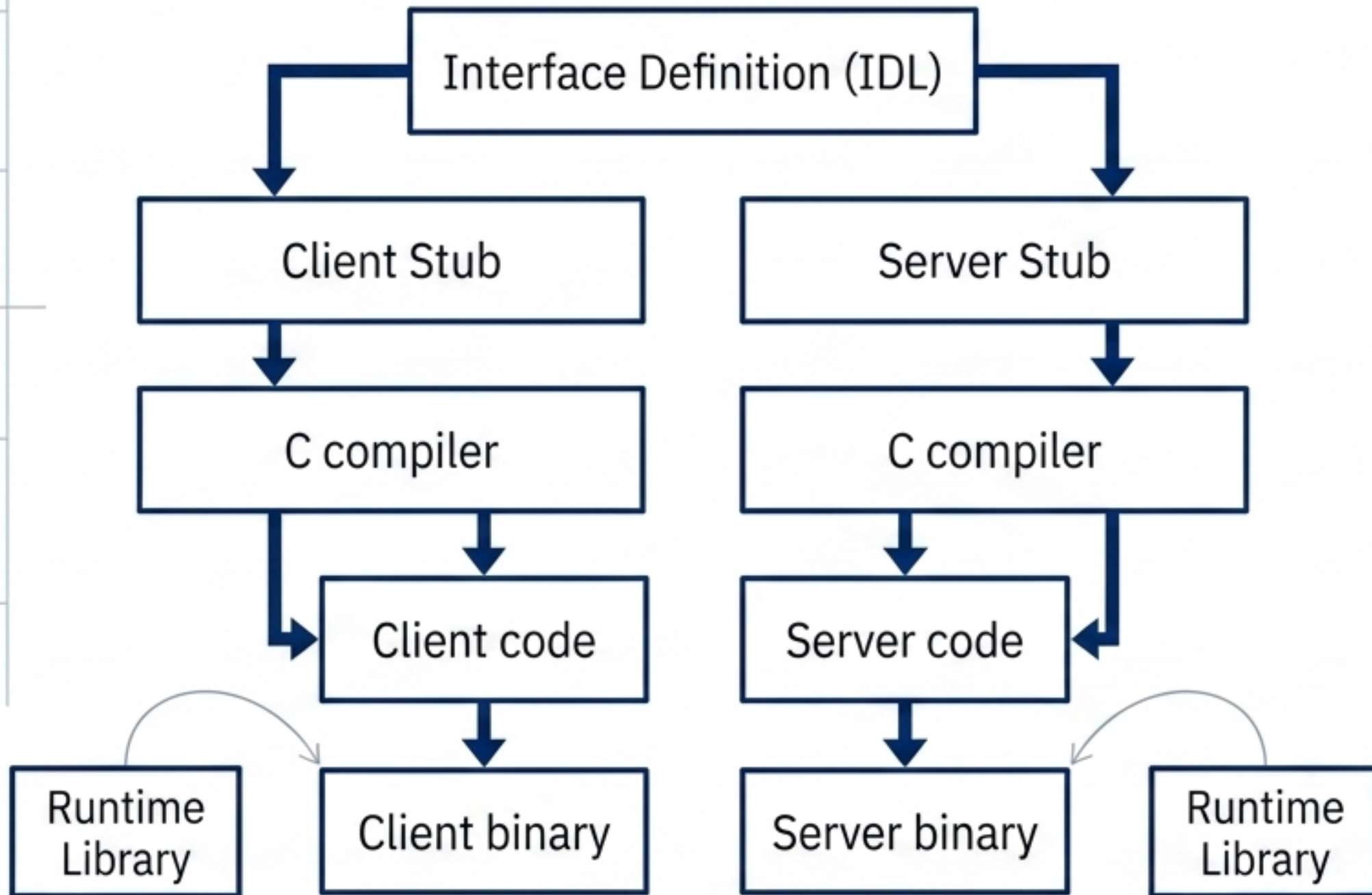


One-Way (Asynchronous)



Automating Complexity

Developers concentrate only on logic. The RPC system generates the boilerplate.



Interface Definition Language (IDL): Establishes the universal contract between systems.



Generators: Automatically compile the IDL into local client and server stubs.



Runtime Libraries: Provide the underlying network execution support seamlessly.

From Procedures to Objects

Remote Method Invocation (RMI)



RPC was built exclusively for procedural programming models.

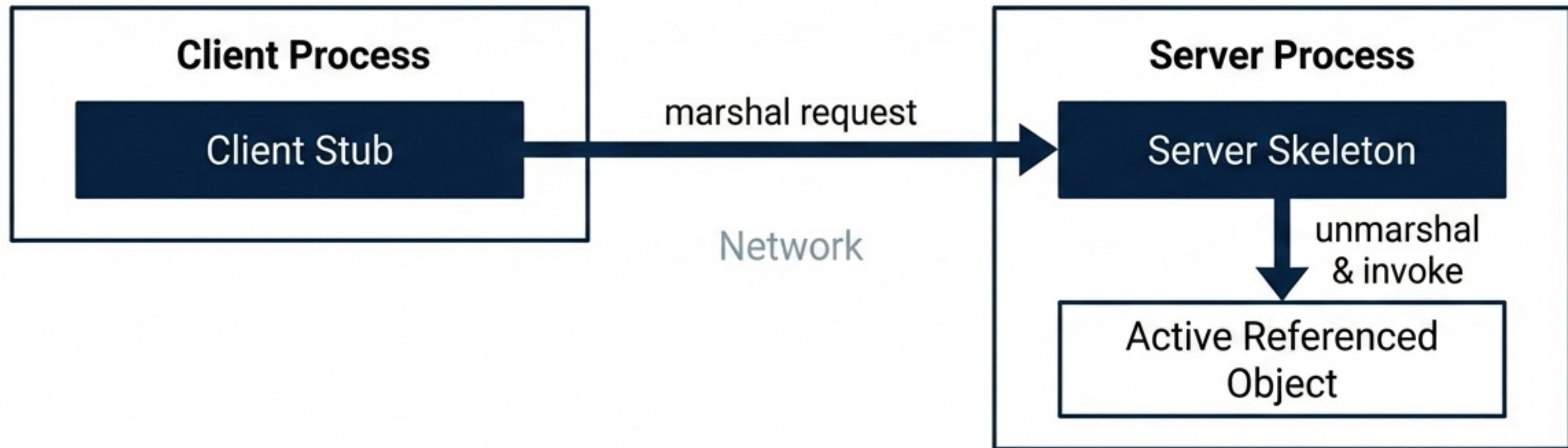


Remote Method Invocation (RMI) adapts this concept for object-oriented paradigms.



Clients **invoke methods directly on encapsulated remote objects**, necessitating new state handling.

The RMI Mechanism



Stubs & Skeletons

Client uses a stub to pack data; Server uses a skeleton to unmarshal and route the method call.

Object Activation

Server ensures the referenced object is active (loaded into memory) before invocation occurs.

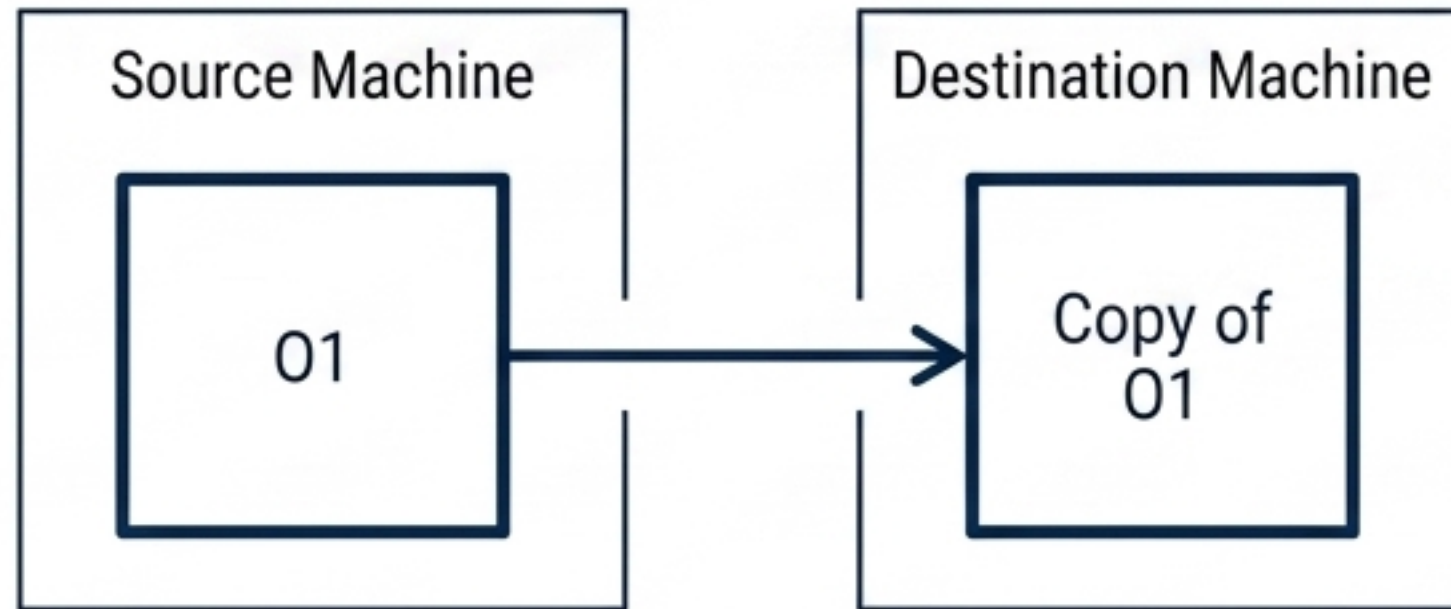
Recursion

If a request contains an object reference, the invocation is applied recursively (the server acts as a client).

RMI Parameter Passing

Handling stateful objects requires new semantics.

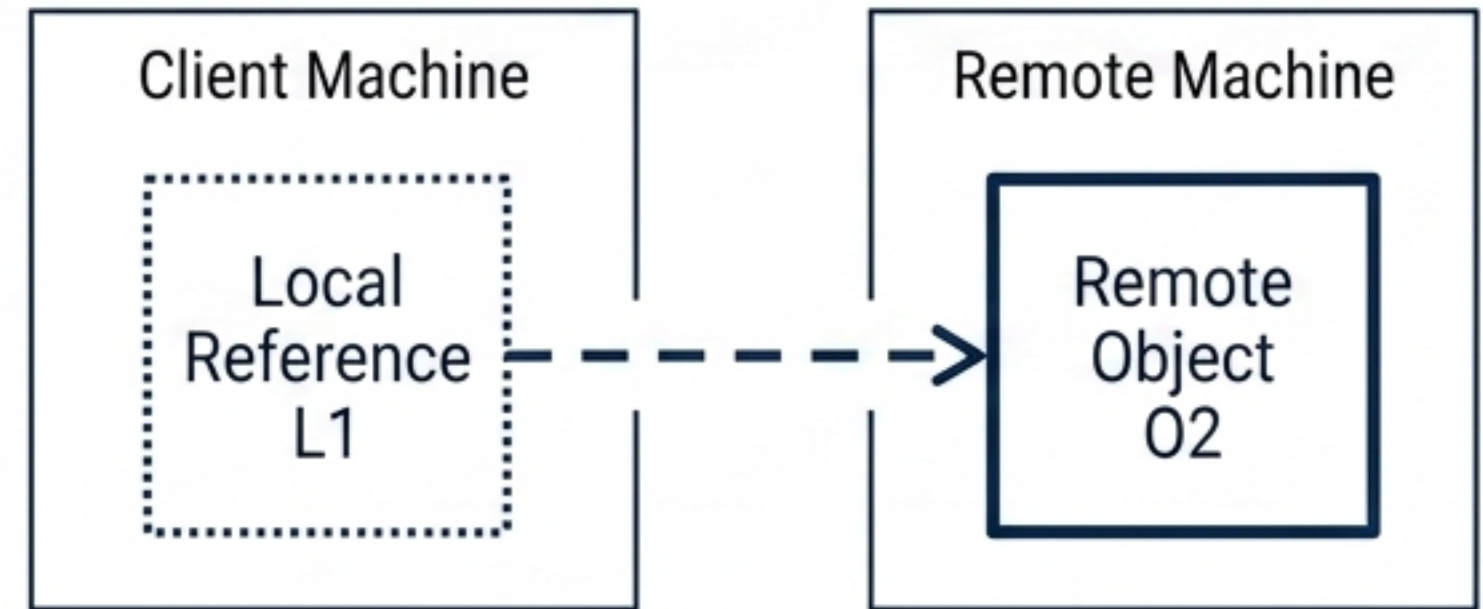
Object-by-Value



Marshals the entire object state and methods.
Creates a complete local copy.

Warning: Introduces complex state synchronization problems.

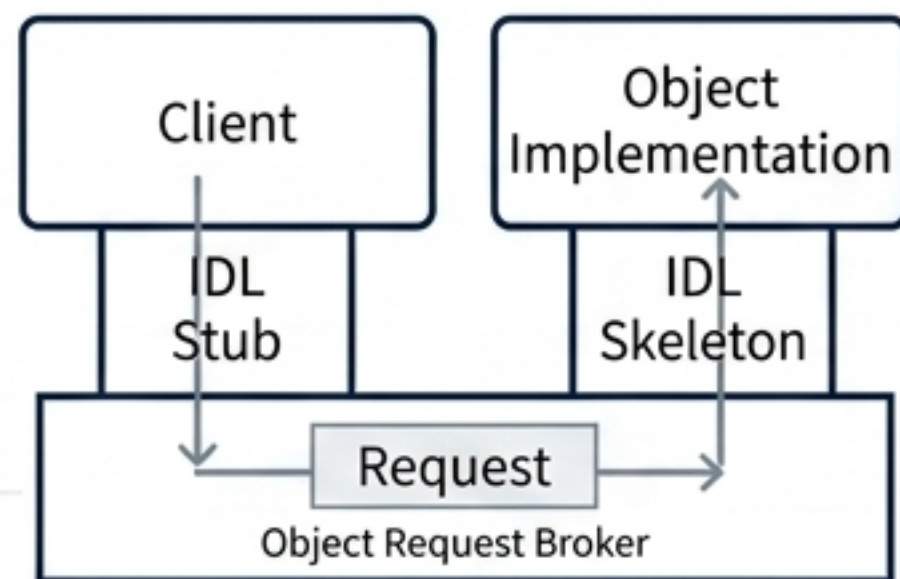
Object-by-Reference



Passes a system-wide identifier containing
the server address, port, local object ID, and
protocol.

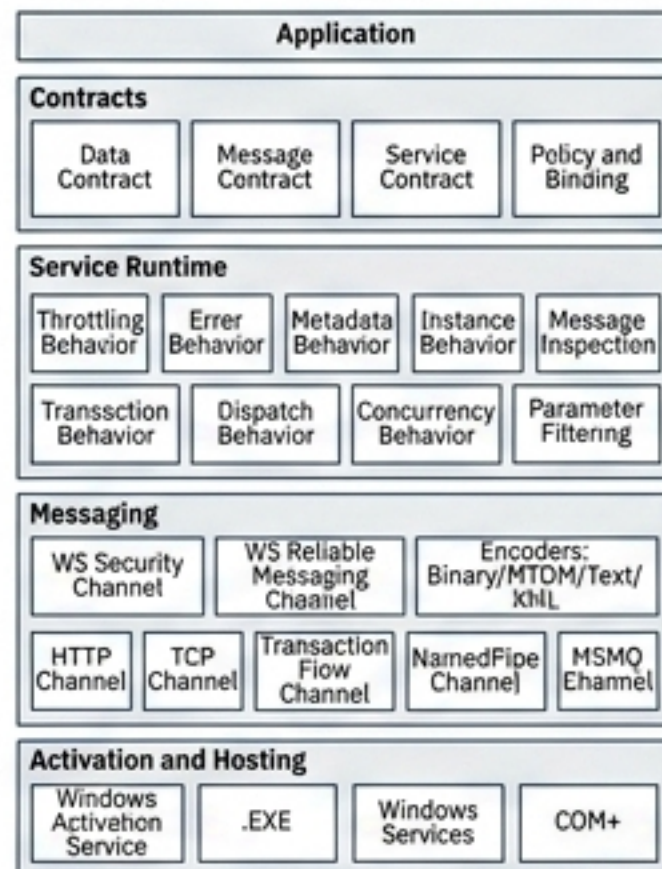
The Technology Landscape

Real-world implementations of remote invocation.



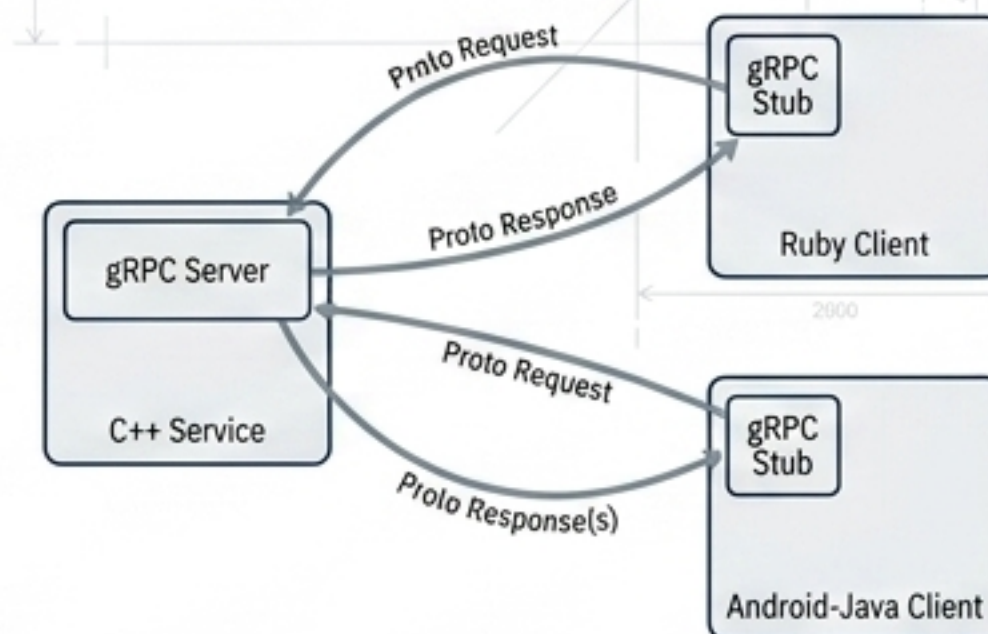
CORBA

Legacy standard. Highly multi-language (COBOL, Ada, Java). Still operational in older enterprise systems.



WCF

Microsoft ecosystem. Web-services based. Deeply integrated with .NET languages and Windows.



gRPC

Modern standard. Google-developed. Language independent. Uses highly efficient binary ProtoBuf over HTTP/2.

The Abstraction Journey

The pursuit of seamless distributed computing.

